

1. Value Creation & Core Problem

Voice interfaces frequently fail because they treat every interaction as an isolated event. A user who dictates a status update in Slack, a draft in Google Docs, and an email in Outlook must repeatedly re-specify formatting preferences, key project names, and team context. Kivi eliminates this repetition by building a durable, cross-application understanding of the user's ongoing work. Real value is created when voice interaction shifts from mechanical speech-to-text to context-aware task completion.

2. The Interaction Boundary: Dictation vs. Co-Pilot

Kivi maintains a clear boundary between its two operational modes:

- **Ordinary Dictation:** Users speak to output text directly into target desktop applications. Here, semantic memory remains completely passive. Dictation never injects unexpected styling or autonomous alterations, preserving predictable transcription while serving as a passive extraction source.
- **Hey Kivi:** An interactive co-pilot mode triggered explicitly by the user. Hey Kivi retrieves semantic memory to execute targeted tasks (e.g., summarizing past drafts or reformatting text). Hey Kivi operates with strict co-pilot autonomy: it retrieves memory to propose outputs, but external tool executions require explicit user confirmation.

3. Memory Taxonomy & Explicit Rejection

Kivi organizes semantic memory into three explicit categories:

- **Factual Memory:** Enduring entity names, team structures, and active project acronyms.
- **Episodic Memory:** Time-bound event logs, recent voice notes, and specific drafting decisions.
- **Preference Memory:** App-scoped formatting rules, tone parameters, and structural outputs.

Explicit Rejection: We deliberately reject acoustic sentiment analysis and emotional inferencing. Kivi will never attempt to infer user stress, pitch, or mood from spoken audio. Inferring emotion introduces unpredictable bias and invasive surveillance. Kivi remembers what a user explicitly states and requests, never how they feel.

4. Deterministic State, Provenance, & No Private Clutter

Kivi pairs probabilistic LLM generation with deterministic state management. All database updates, memory overrides, decay schedules, and retrieval rules follow auditable code logic. To avoid storing private clutter, Kivi does not retain full-text audio or transcript dumps in long-term memory. Instead, it persists lightweight key-value facts linked to source metadata pointers (e.g., [App: Slack](#), [Timestamp: Aug 12](#), [Record_ID: 89f2](#)). Every answer provided by Hey Kivi is fully traceable to its original interaction source.

5. Memory Lifecycle & User Agency

- **Decay Rules:** Episodic memories carry a 30-day expiration period. The decay clock resets whenever the episodic context is re-referenced in a new dictation or successfully retrieved by Hey Kivi. Factual and Preference memories persist until manually altered or overridden.
- **Scoped Overrides:** Preferences are scoped to specific target applications by default. When a user states a contradictory rule (e.g., *"Format Slack updates as bullet points"*), the system resolves the conflict by updating the existing record for that specific scope.
- **User Control:** Users retain full sovereignty through a native **Memory Viewer** interface to inspect, edit, or delete any stored state without administrative overhead.